

M1 - Noyau 4I401

Révision - Décembre 2017

I. SYNCHRONISATION

Nous voulons offrir la fonction système `int sum_sync(int id, int n, int val)` qui réalise une somme synchronisée parmi n processus qui appellent la fonction avec le même `id` (identifiant de l'opération de la somme en cours). La fonction rend la somme des valeurs passées dans le paramètre `val` par chacun des n processus appelants.

Exemple : supposons 3 processus.

(même ID pour tout les processus)

P_1 $\downarrow$ n $\downarrow$ val
...
`sum_sync (1, 3, 2);`
`sum_sync (2, 3, 3);`

P_2 $\downarrow$
...
`sum_sync (1, 3, 4);`
`sum_sync (2, 3, 2);`

P_3 $\downarrow$
...
`sum_sync (1, 3, 1);`
`sum_sync (2, 3, 4);`

Pour les 3 processus, le premier et le deuxième appels à `sum_sync` renvoient 7 et 9 respectivement.

Nous considérons que les programmes utilisent correctement `sum_sync(int id, int n, int val)` : pour un `id` donné, il y a toujours n processus qui appellent la fonction et n est passé comme deuxième paramètre par tous les processus appelants. De plus, un processus n'appelle jamais deux fois la fonction avec une même valeur d'`id`.

Si un processus s'endort, sa priorité de réveil est `P_WAIT`.

I.1.

Donnez le code de l'appel système `_sum_sync` appelé par la fonction `sum_sync(int id, int n, int val)` en spécifiant les champs que vous allez ajouter dans les structures du noyau et sur quelle adresse les processus vont s'endormir.

II. SYNCHRONISATION

Dans cet exercice, on veut ajouter au noyau la possibilité d'attendre la fin d'une entrée/sortie (E/S) asynchrone via une fonction interne `fin_ES`.

II.1.

Donnez l'implémentation d'une fonction noyau "`fin_ES(dev, blkno)`" qui bloque un processus si une E/S asynchrone est en cours et ce jusqu'à la fin de cette E/S.

II.2.

Modifier le code original du noyau pour réveiller un processus en attente dans la fonction `fin_ES`. Vous indiquerez clairement la/les fonction(s) ou structure(s) modifiée(s), ainsi que la modification.

Attention : votre solution devra éviter toute perte de temps si aucun processus n'est en attente sur cet évènement.

III. ECRITURE FONCTION SYSTEME

Nous voulons offrir fonction système `int adopt_grandchild ( )` qui permet d'adopter les fils des fils du processus. Autrement dit, les petit-fils du processus appelant deviennent ses fils directs en cessant d'être les fils de leur père d'origine. L'appel doit rendre le nombre de fils adoptés par le processus appelant.

Nous considérons dans un premier temps que lors de l'appel à `adopt_grandchild ( )`, aucun des processus pères d'origine étaient en train d'attendre la terminaison d'un des leurs "ex-fils".

III.1.

Donnez le pseudo-code de l'appel système `int _adopt_grandchild ( )` appelée par la fonction système `int adopt_grandchild ( )`.

Nous considérons maintenant que lors de l'appel à `int _adopt_grandchild ( )`, il peut y avoir des processus fils de l'appelant qui attendait la terminaison d'un des leurs "ex-fils".

III.2.

Modifiez en conséquence l'appel système `_adopt_grandchild`. Cet appel ne doit pas essayer de réveiller un processus fils qui ne se trouve pas bloqué en attente d'un de leur "ex-fils".

Note: nous considérons que lorsque le processus fils sera débloqué dans la fonction `wait (int* status)`, celle-ci terminera en indiquant que le processus n'a pas de fils.

IV. ECRITURE FONCTION SYSTEME

On souhaite offrir la fonction système `wait_parent(void)` où processus appelant doit se bloquer tant que son père ne s'est pas terminé. L'appelant s'endort alors à la priorité PWAIT. Si l'appelant n'a plus de père au moment de l'appel (c'est à dire qu'il a été adopté par `init`, le processus 1) la fonction doit retourner directement 0 sinon elle retourne 1 au réveil.

IV.1.

Donnez le code de l'appel système `_wait_parent` appelé par la fonction `int wait_parent()` ainsi que les modifications éventuelles à faire dans le code de exit donné en annexe (dans ce cas vous indiquerez uniquement les lignes à changer/ajouter). Vous pouvez si nécessaire ajouter des champs supplémentaires dans la structure `proc`.

IV.2.

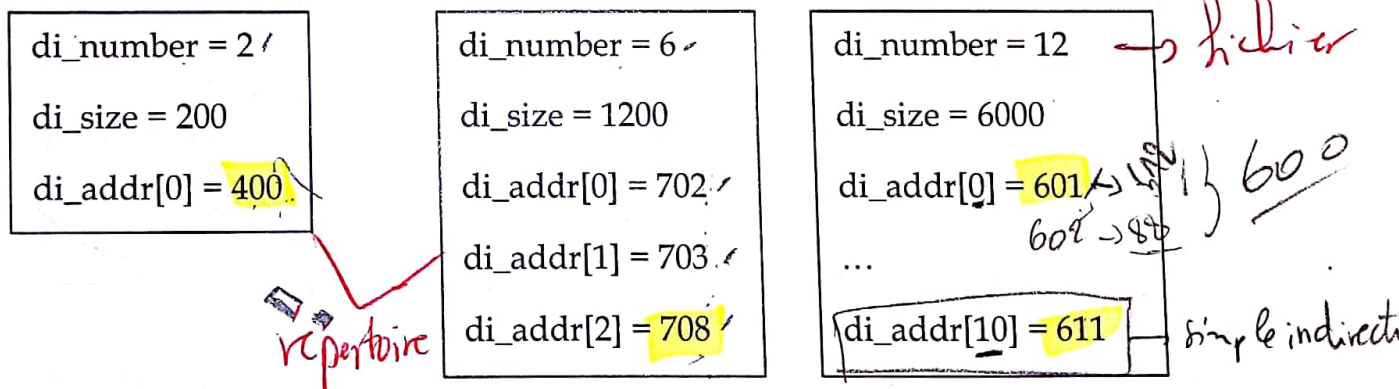
Modifiez la fonction `_wait_parent()` pour que l'appelant soit réveillé si le père ne s'est pas terminé au bout de MAXTICK ticks. Vous utiliserez la fonction `timeout`.

V. SYSTEME DE FICHIER

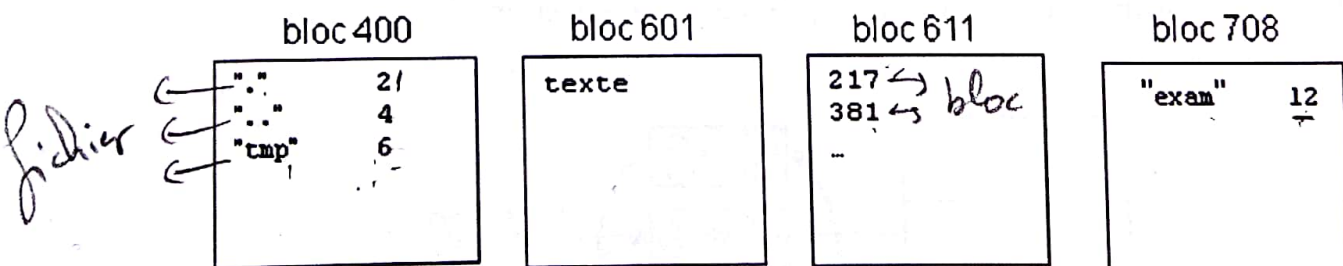
On considère la partition numéro ^{numéro device.} 0 avec des blocs de 512 octets. Le superbloc comprend les blocs 1 et 2. La table des inodes commence à partir du bloc 3 inclus : l'inode 0 se trouve à l'offset 0 du bloc 3. Les inodes ont une taille de 128 octets.

On considère un processus P. Son répertoire courant, indiqué par « `cur_dir` » désigne l'inode numéro 2. Cet inode est la seule chargée en mémoire.

Soit la configuration suivante des inodes sur disque :



Les inodes 2 et 6 sont de type répertoire et l'inode 12 de type fichier. On considère que les blocs 702 et 703 ne contiennent pas de données intéressantes pour l'examen et que les contenus des blocs 400, 601, 611 et 708 sont les suivants :



Handwritten notes and calculations:

- `/tmp / exam.`
- Path calculation: `./tmp / Exam.` with block numbers 3 and 4, leading to "chemin d'accès".
- Calculation: $217 \times 512 = 111104$.

V.1.

V.1.
Décrivez le chemin d'accès au fichier "exam" à partir du répertoire courant. Dans la suite, on appelle chemin_fichier ce chemin d'accès. "`./tmp/exam`"
Que représentent les nombres 217 et 381 contenus dans le bloc 611 ? → *bloc de*

bloc de données
~~simple~~ ~~indirection~~

V.2.

Donnez l'expression de la localisation des inodes sur disque en fonction des caractéristiques de la partition (position de l'inode 0, nombre d'inode par bloc). Précisez sur quels blocs physiques les inodes 2, 6 et 12 sont placés.

V.3.

Décrivez la suite des actions effectuées (appel de fonctions, numéro de blocs accédés, recherche de motifs, extraction de données de blocs, écritures dans la table d'inode) lors de l'exécution de l'instruction :

```
fd = open (« chemin_fichier », O_RDONLY) ;
```

V.4. *by*

A la suite de cette instruction d'ouverture, P exécute les instructions suivantes :

```
read(fd, mon_buff, 10); → logique offset = 10.
lseek(fd, 10*512, CUR_SEEK);
read(fd, mon_buff, 10);
```

Décrivez l'évolution de `fd->f_offset` et la suite d'accès aux blocs induite par cette séquence d'exécution.

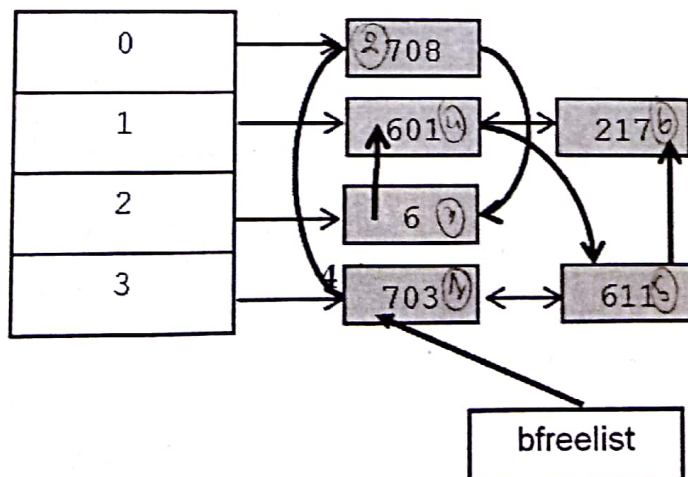
On suppose que le programme P exécute des instructions qui engendrent la séquence d'accès en lecture ou écriture aux blocs suivants :

$W(6), R(5), R(200), R(201), R(6), R(611), W(381).$

Le système utilise un buffer cache composé de 6 buffers ; l'ensemble des buffers est haché en 4 sous listes (on considère la fonction de hash est $\text{blkno} + \text{dev} \% 4$). L'état du buffer cache avant les accès est représenté sur le schéma ci-dessous. Pour simplifier on n'indique pas les numéros de partition.

V.5.

Représentez l'état du buffer cache après exécution de la séquence d'accès de la question I.5. Combien d'E/S cette séquence a-t-elle généré ?



ys-sysnc () {

REV 2017

p-pid

struct proc *p = u.u - proc - p;
int i, n;
cpt = 0; sum = 0;

p₁ → $\boxed{\begin{matrix} p - pid \\ \textcircled{2} \end{matrix}}$

p → pid = u.u - arg[0];

p₂ → p - pid =

p → p-val = u.u - arg[2]; → pointer jumps.

n = u.u - arg[1];

for (i=0 ; i < NPROC ; i++) {

if (proc[i] → p-id == p-id)

cpt++; // incrementer le compteur

sum += proc[i] → p-val;

}

if (cpt == n) {

cpt = 0;

}

for (i=0 ; i < NPROC && (cpt < n) ; i++) {

if (proc[i] → p-id == p → p → p-id) &&

(proc[i] → p-id != p → p → p-id)) {

cpt++;

proc[i] → p-sum = sum;

wakeup (& proc[i] → p-pid)

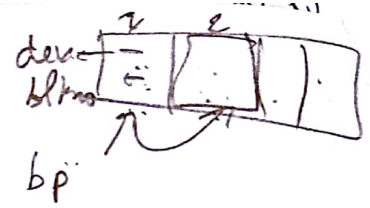
}

}

else sleep (p → id, Pwait);

} u.u - arg[0] = p → p-sum; →

(2) Fin E/S.



Fin E/S (dev, blkno) {

int find=0;

bp = bhash (dev, blkno); → fonction de hachage.

for (bp = dp → b-forw, bp != dp; bp = dp → b-forw)

if ((bp → b-blkno != blkno) || (bp → b-dev != dev))

continue;

find = 1;

break;

if (find) {

sigpl

spl (BDIM+HB)

if ((bp → b-flag & B-async) && !=

(bp → b-flag & B-DONE)) {

bp → b-flag != B-wait-io;

sleep (bp+1; PRIBIO);

}

spl (5);

}

2) lorsque une interruption $\rightarrow$ on appelle IDONE.

```
iodone (bp) {
```

```
    bp  $\rightarrow$  b-flag |= B-DONE // = 1.
```

```
    if (bp  $\rightarrow$  b-flag & B-Async != 0) {
```

```
        bclear (bp);
```

```
    else
```

```
        wakeup (bp);
```

```
    }
```

```
    if (bp  $\rightarrow$  b-flag & B-wait-IO)
```

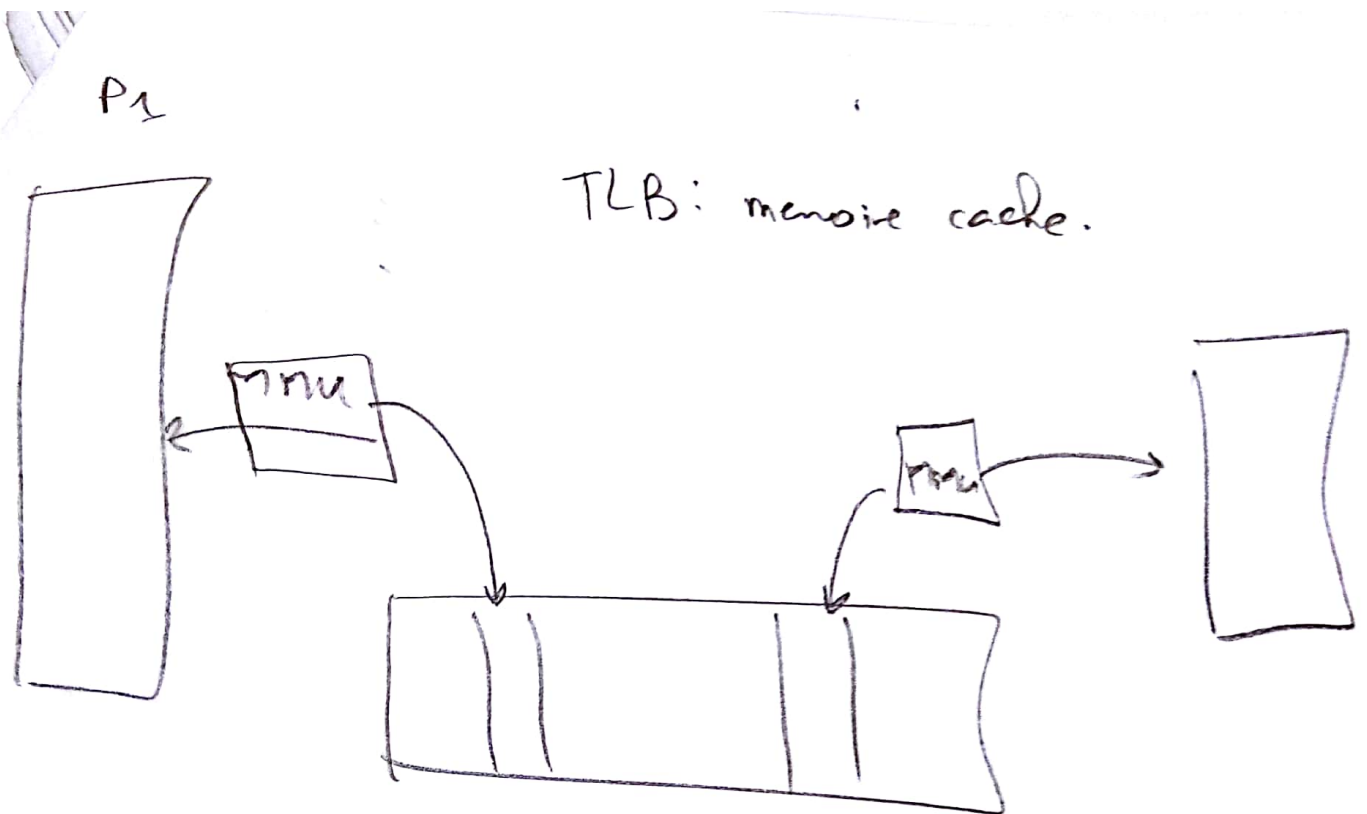
```
    {
```

```
        bp  $\rightarrow$  b-flag & = ~ B-wait-IO ;
```

```
        wakeup (bp++);
```

```
    } -
```

```
}
```



mnu: lien @ physique. et @ virtuel.

traduire virtuel en physique.

TLB contient les @